

Content

Content	3
Chapter 1. Introduction & Objectives	4
1.1 Introduction	4
1.2 Objective:	4
Chapter 2. System Modelling	5
2.1 Mathematical Equation	5
2.2 Assumptions	6
Chapter 3. Feedback Control Design	7
3.1 Closed-Loop Implementation	7
3.2 Controller Theory	8
3.2.1 P Controller (Proportional)	8
3.2.2 PI Controller (Proportional-Integral)	9
3.2.3 PD Controller (Proportional-Derivative)	9
3.2.4 PID Controller (Proportional-Integral-Derivative)	10
Chapter 4. MATLAB Implementation	11
Chapter 5. Performance Analysis	14
5.1.1 Plot Comparing All Controllers	14
5.1.2 System Simulation without Controllers	15
5.1.3 System Simulation using P Controller	16
5.1.4 System Simulation using PI Controller	17
5.1.5 System Simulation using PD Controller	18
5.1.5 System Simulation using PID Controller	19
5.2 Performance Discussion	20
Chapter 6. Conclusion	21
References	22

Chapter 1. Introduction & Objectives

1.1 Introduction

Control System is an essential tool in advanced engineering applications, by ensuring efficiency, reliability, and stability for modern systems. Feedback control achieves a major impact for the system performance, reducing the steady state error, increasing time rise, enhancing overshoot, and reducing the settling time. In this project, a mass-spring damper modeled and simulated using matlab and Simulink extension to study and analyse the effect of using different PID controllers on the overall system performance by comparing the results of the system response to the original system before controllers used and measuring the difference between each controller.

1.2 Objective:

This project seeks to analyze a mass-spring damper mechanical control system with actual closed-loop control simulations to determine how controllers may change or improve the operation of the system and identify what type of control architecture or controller will produce the best possible performance out of the system. While the original goal of the project is to create a correctly functioning system, and determine if there are other ways in which the performance of the system control can be optimized such as steady-state error, time response, delay time, and settling time for more stable performance faster and more efficiently than before.

Chapter 2. System Modelling

2.1 Mathematical Equation

The Mass-Spring-Damper system is governed by Newton's Second Law of Motion. Applying the law to the mass m subject to an applied force $F(t)$:

$$m \cdot \ddot{x}(t) + b \cdot \dot{x}(t) + k \cdot x(t) = F(t)$$

Where $x(t)$ is the displacement of the mass, $\dot{x}(t)$ is velocity, $\ddot{x}(t)$ is acceleration, $F(t)$ is the external forcing input, and the system parameters are:

- $m = 1 \text{ kg}$ (mass)
- $b = 0.5 \text{ N}\cdot\text{s/m}$ (viscous damping coefficient)
- $k = 1 \text{ N/m}$ (spring stiffness constant)

Taking the Laplace Transform of both sides (assuming zero initial conditions):

$$m \cdot s^2 \cdot X(s) + b \cdot s \cdot X(s) + k \cdot X(s) = F(s)$$

$$X(s) \cdot [ms^2 + bs + k] = F(s)$$

The open-loop Transfer Function $G(s)$ is therefore:

$$G(s) = X(s) / F(s) = 1 / (ms^2 + bs + k)$$

Substituting the numerical values $m = 1$, $b = 0.5$, $k = 1$:

$$G(s) = 1 / (s^2 + 0.5s + 1)$$

This is a standard second-order transfer function. The natural frequency ω_n and damping ratio ζ can be identified as:

$$\begin{aligned}\omega_n &= \sqrt{k/m} = \sqrt{1/1} = 1 \text{ rad/s} \\ \zeta &= b / (2\sqrt{mk}) = 0.5 / (2 \cdot \sqrt{1 \cdot 1}) = 0.25\end{aligned}$$

Since $0 < \zeta < 1$ ($\zeta = 0.25$), the uncontrolled system is underdamped, meaning the step response will exhibit decaying oscillations before reaching steady state. With unity negative feedback but no controller, the closed-loop transfer function is:

$$T(s) = G(s) / (1 + G(s)) = 1 / (s^2 + 0.5s + 2)$$

The DC gain of this closed-loop system is $T(0) = 1/2 = 0.5$, meaning the output settles at 0.5 rather than 1.0. A steady-state error of 50%. This motivates the need for controllers with integral action.

In MATLAB, the transfer function is defined with the following code:

```
m = 1; b = 0.5; k = 1;  
num = [1];  
den = [m b k]; % den = [1 0.5 1]  
G = tf(num, den);
```

2.2 Assumptions

The following assumptions are used to obtain the above mathematical model and ensure that simulation results are relevant:

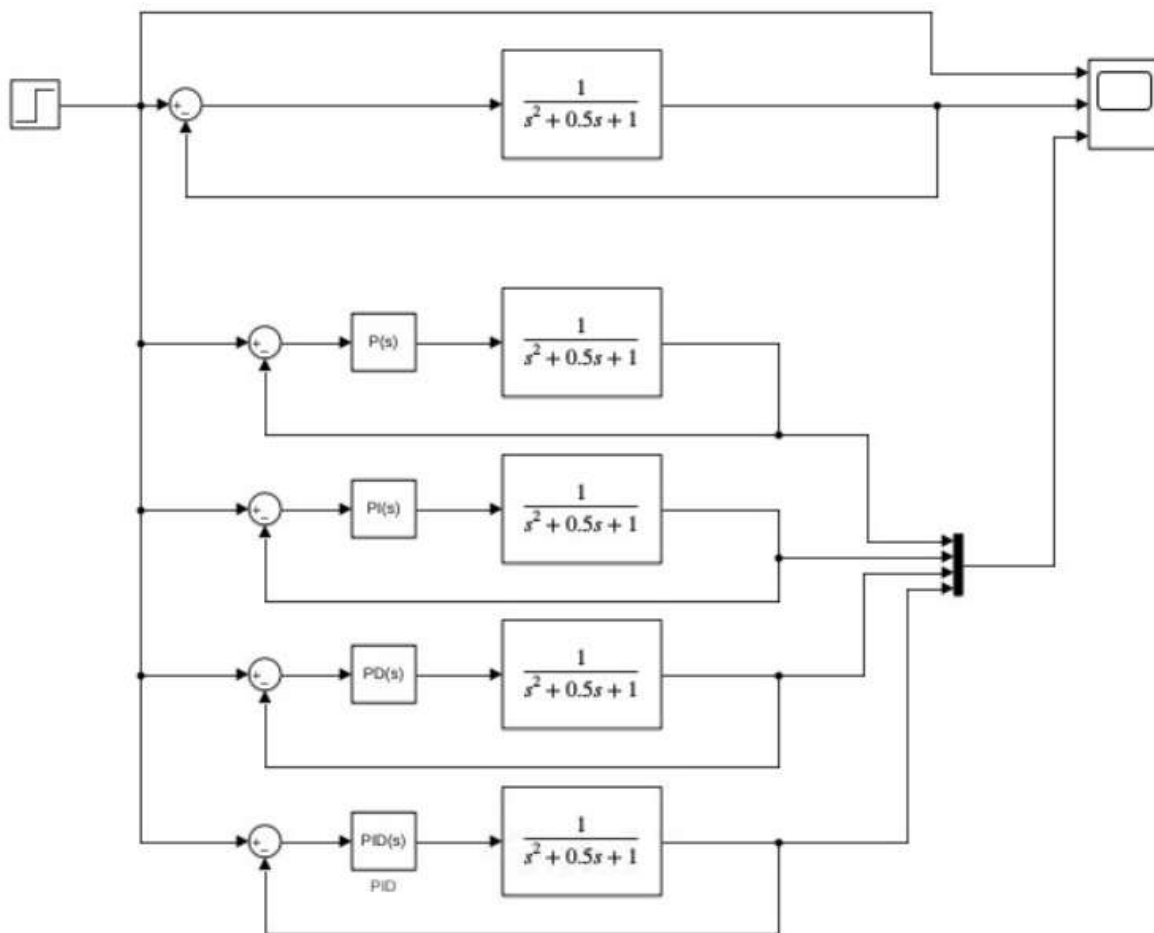
- **Linear System Behavior:** All three elements (mass, spring, and damper) are assumed to be linear and time-invariant (LTI). The spring force is purely Hookean ($F = kx$) and the damper force is purely viscous ($F = b\dot{x}$), with no nonlinear effects.
- **Zero Initial Conditions:** At time $t = 0$, both displacement $x(0) = 0$ and velocity $\dot{x}(0) = 0$. This allows the direct application of the Laplace Transform without additional terms.
- **One-Dimensional Motion:** The mass moves only in one translational direction. Rotational dynamics, friction, and gravity are neglected.
- **Rigid Body Assumption:** The mass is treated as a point mass or rigid body, no internal deformation or distributed mass effects are considered.
- **Spring and Damper:** An ideal spring had no mass and an ideal damper had no springiness. The interaction was between Hooke's Law and viscous damping.
- **Unity Feedback:** A gain of 1 was considered on the feedback, which states the sensor measures output with no scaling, noise, or delay.
- **Unit Step:** Every simulation was run with the input signal, $F(s)=1/s$, which is known as a unit step function and is used to measure the system's time-response performance.
- **Continuous Time Domain:** All analyses were conducted in the time domain and there were no effects of sampling or discretisation.

Chapter 3. Feedback Control Design

3.1 Closed-Loop Implementation

A unity-feedback closed-loop control architecture is implemented for this system. In this configuration, the output signal $x(t)$ is fed back and compared with the reference input $r(t)$ to generate an error signal $e(t)$. This error is then processed by the controller $C(s)$ to produce the control action $u(t)$, which drives the system $G(s)$.

The block diagram structure is:



The closed-loop transfer function for any controller $C(s)$ is given by the standard feedback formula:

$$T(s) = C(s) \cdot G(s) / (1 + C(s) \cdot G(s))$$

In MATLAB, the closed-loop systems are implemented using the `feedback()` function:

```
sys = feedback(G, 1);      % No controller (original system)
sys_p = feedback(C_p * G, 1); % With P controller
sys_pi = feedback(C_pi * G, 1); % With PI controller
sys_pd = feedback(C_pd * G, 1); % With PD controller
sys_pid = feedback(C_pid * G, 1); % With PID controller
```

Unity feedback (second argument = 1) means the sensor gain is $H(s) = 1$. The step responses are then obtained over a simulation time of 0 to 20 seconds (40 secs if needed) with a step size of 0.01 seconds, providing sufficient resolution to capture all transient dynamics.

3.2 Controller Theory

Classical PID control theory provides a systematic framework for modifying the open-loop dynamics to achieve desired closed-loop performance. The general PID controller in the time domain is:

$$u(t) = K_p \cdot e(t) + K_i \int e(\tau) d\tau + K_d \cdot de(t)/dt$$

In the Laplace (frequency) domain, the PID transfer function is:

$$C(s) = K_p + K_i/s + K_d \cdot s$$

The four controller variants studied in this project are special cases of this general structure, each activating a different subset of the three terms.

3.2.1 P Controller (Proportional)

The Proportional controller produces a control action proportional to the instantaneous error signal. Its transfer function is:

$$C_P(s) = K_p = 5$$

The P controller with $K_p = 5$ is implemented in MATLAB as:

```
C_p = pid(5, 0, 0);
```

Effect on system performance:

- Increases the loop gain, which reduces (but does not eliminate) steady-state error for a Type 0 system.
- Speeds up the transient response and lowers rise time compared to the uncontrolled system.
- May introduce or increase overshoot due to the increased gain.
- Does not add poles or zeros to the open-loop transfer function; only scales the loop gain.
- **Limitation:** Steady-state error persists because there is no integral action. For the Type 0 system, the steady-state output will be $K_p \cdot G(0) / (1 + K_p \cdot G(0)) = 5 / (1 + 5) = 5/6 \approx 0.833$.

3.2.2 PI Controller (Proportional-Integral)

The PI controller adds an integral term to the proportional action. Its transfer function is:

$$C_{PI}(s) = K_p + K_i/s = 3 + 1/s$$

In MATLAB:

```
C_pi = pid(3, 1, 0);
```

Effect on system performance:

- The integral term (K_i/s) adds a pole at the origin, increasing the system type by 1. This guarantees zero steady-state error for a step input.
- Eliminates the offset that a P-only controller leaves, enabling perfect tracking of constant references.
- The integral action is slower to respond, which can increase rise time and settling time compared to P control.
- Can introduce additional phase lag, potentially degrading transient response (more overshoot) if not properly tuned.
- **Trade-off:** Improved steady-state accuracy at the cost of potentially slower and more oscillatory transient response.

3.2.3 PD Controller (Proportional-Derivative)

The PD controller augments proportional action with a derivative term. Its transfer function is:

$$C_{PD}(s) = K_p + K_d \cdot s = 5 + 2.3s$$

In MATLAB:

```
C_pd = pid(5, 0, 2.3);
```

Effect on system performance:

- The derivative term ($K_d \cdot s$) adds a zero to the open-loop transfer function, introducing phase lead.
- Phase lead acts as a predictive element. It anticipates future errors based on the rate of change of the current error.
- Significantly reduces overshoot and improves damping of the transient response.
- Can improve stability margins and allow higher proportional gains.
- **Limitation:** Like the P controller, PD has no integral action and therefore cannot eliminate steady-state error for a Type 0 system. It is sensitive to measurement noise since differentiation amplifies high-frequency components.

3.2.4 PID Controller (Proportional-Integral-Derivative)

The PID controller combines all three actions, representing the most complete form of classical feedback control:

$$C_PID(s) = K_p + K_i/s + K_d \cdot s = 3 + 2.7/s + 3.5s$$

In MATLAB:

```
C_pid = pid(3, 2.7, 3.5);
```

Effect on system performance:

- Combines the benefits of all three terms: fast response (P), zero steady-state error (I), and improved damping (D).
- The integral term eliminates offset; the derivative term counteracts the phase lag introduced by the integral term.
- Properly tuned, the PID achieves the best overall balance of rise time, settling time, overshoot, and steady-state accuracy.
- In this project, the PID controller is tuned with $K_p = 3$, $K_i = 2.7$, $K_d = 3.5$ to deliver near-optimal performance.
- **Result:** The PID controller provides the best overall closed-loop performance among all controllers tested.

Chapter 4. MATLAB Implementation

The entire simulation was carried out in MATLAB. The script was used to define the system variables, the transfer function of the system, design all four controllers (using `pid()` command) and then assemble the closed loop systems using unity feedback. The step response of each closed loop system was then plotted. The entire MATLAB code has been included below:

```
clear; clc; close all;

% 1. Assumptions used in the modeling process
% Modeling a mass-spring-damper system
m = 1;    % Mass (kg)
b = 0.5;  % Damping coefficient (N-s/m)
k = 1;    % Spring constant (N/m)

% 2. Mathematical model (Transfer Function)
num = [1];
den = [m b k];
G = tf(num, den); % system transfer function

% 3. Define Controllers
C_p = pid(5, 0, 0);    % P
C_pi = pid(3, 1, 0);   % PI
C_pd = pid(5, 0, 2.3); % PD
C_pid = pid(3, 2.7, 3.5); % PID

% 4. Closed-Loop Systems
sys = feedback(G, 1);
sys_p = feedback(C_p * G, 1);
sys_pi = feedback(C_pi * G, 1);
sys_pd = feedback(C_pd * G, 1);
sys_pid = feedback(C_pid * G, 1);

% 5. Time vector
t = 0:0.01:40;

% --- Get step responses ---
[y_sys, t] = step(sys, t);
[y_p, ~] = step(sys_p, t);
[y_pi, ~] = step(sys_pi, t);
[y_pd, ~] = step(sys_pd, t);
[y_pid, ~] = step(sys_pid, t);
```

```

ref = ones(size(t)); % Reference signal

% ----- Original System Figure -----
figure;
plot(t, y_sys, 'y', t, ref, 'k--', 'LineWidth', 1.5);
title('Original System Response');
xlabel('Time'); ylabel('Amplitude');
legend('Original System', 'Reference');
grid on;

% --- Combined Plot ---
figure;
plot(t, y_sys, 'y', ...
     t, y_p, 'r', ...
     t, y_pi, 'g', ...
     t, y_pd, 'm', ...
     t, y_pid, 'b', ...
     t, ref, 'k--', 'LineWidth', 1.5);

title('Comparative Output Response Plots');
xlabel('Time');
ylabel('Amplitude');
legend('Original System', 'P', 'PI', 'PD', 'PID', 'Reference');
grid on;

%% ----- Individual Figures -----

% P Controller
figure;
plot(t, y_sys, 'y', t, y_p, 'r', t, ref, 'k--', 'LineWidth', 1.5);
title('P Controller vs Original System');
xlabel('Time'); ylabel('Amplitude');
legend('Original', 'P Control', 'Reference');
grid on;

% PI Controller
figure;
plot(t, y_sys, 'y', t, y_pi, 'g', t, ref, 'k--', 'LineWidth', 1.5);
title('PI Controller vs Original System');
xlabel('Time'); ylabel('Amplitude');
legend('Original', 'PI Control', 'Reference');
grid on;

% PD Controller
figure;
plot(t, y_sys, 'y', t, y_pd, 'm', t, ref, 'k--', 'LineWidth', 1.5);
title('PD Controller vs Original System');
xlabel('Time'); ylabel('Amplitude');

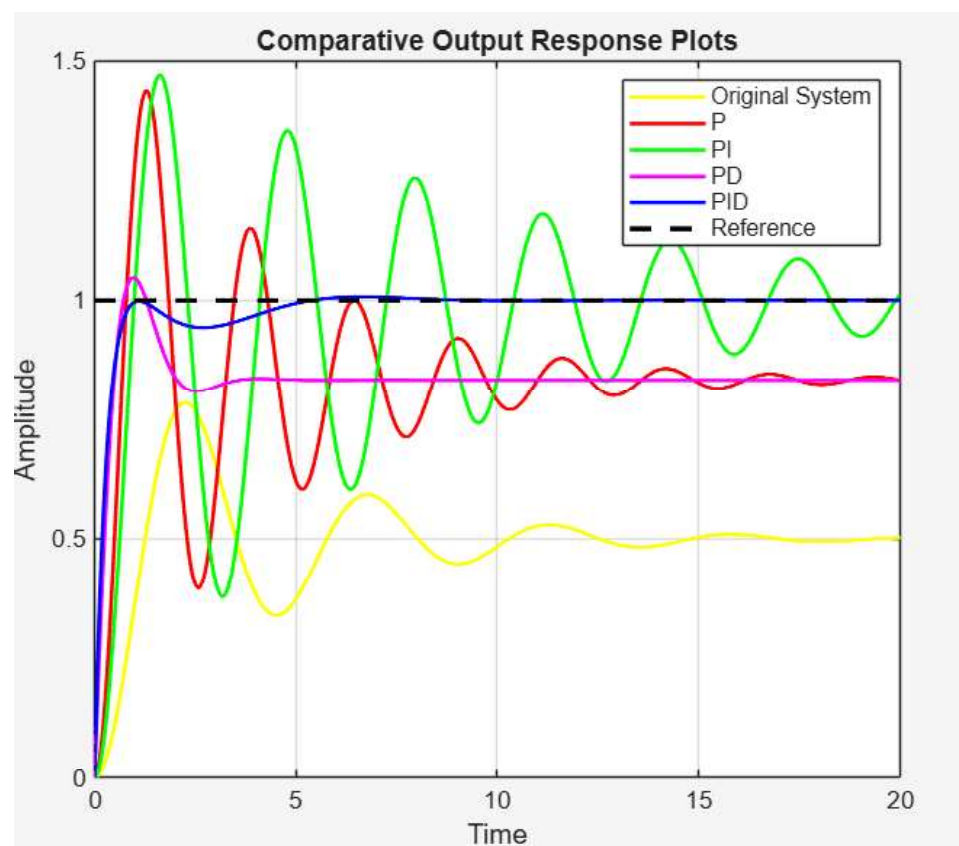
```

```
legend('Original', 'PD Control', 'Reference');  
  
grid on;  
  
% PID Controller  
figure;  
plot(t, y_sys, 'y', t, y_pid, 'b', t, ref, 'k--', 'LineWidth', 1.5);  
title('PID Controller vs Original System');  
xlabel('Time'); ylabel('Amplitude');  
legend('Original', 'PID Control', 'Reference');  
grid on;  
  
% 8. Performance Analysis  
fprintf('--- Performance Analysis ---\n');  
fprintf('Original System:\n'); disp(stepinfo(sys));  
fprintf('P Controller:\n'); disp(stepinfo(sys_p));  
fprintf('PI Controller:\n'); disp(stepinfo(sys_pi));  
fprintf('PD Controller:\n'); disp(stepinfo(sys_pd));  
fprintf('PID Controller:\n'); disp(stepinfo(sys_pid));
```

Chapter 5. Performance Analysis

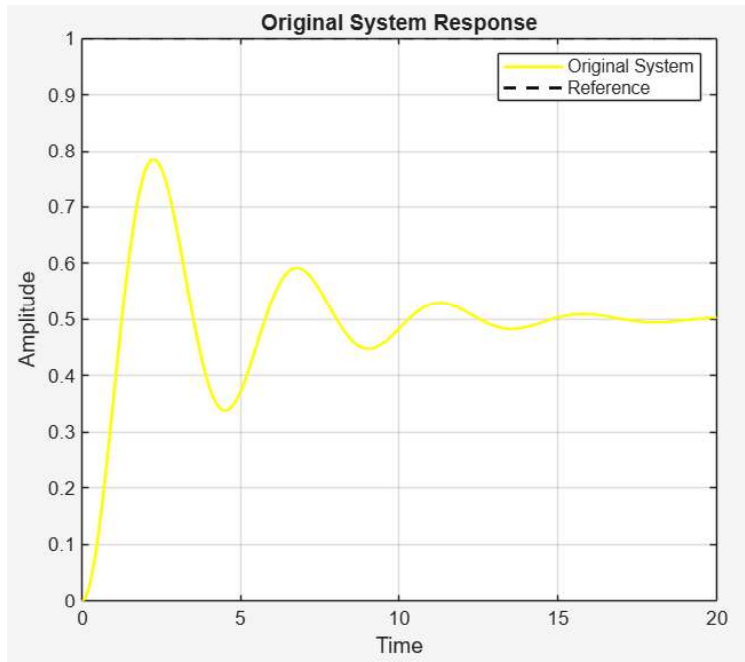
5.1.1 Plot Comparing All Controllers

The first figure created by the MATLAB script shows the step responses of all five systems (no control, P, PI, PD, and PID controllers), together with the unit step reference input. This is the simplest and most intuitive form of visualization showing the effect of each controller on the system response.



5.1.2 System Simulation without Controllers

The original closed-loop system is formed by placing the system $G(s)$ in a unity negative feedback loop without any additional controller ($C(s) = 1$). The closed-loop transfer function becomes:



$$T_{\text{orig}}(s) = G(s) / (1 + G(s)) = 1 / (s^2 + 0.5s + 2)$$

Performance characteristics of the original system:

- Steady-State Value ≈ 0.5 The output settles at 0.5, not 1.0. There is a 50% steady-state error due to the finite DC gain of the closed-loop system.
- The system is underdamped ($\zeta \approx 0.25$ in the closed-loop), producing a slightly oscillatory step response.
- Rise time is moderate. the system responds within a few seconds but does not reach the full reference.
- The response demonstrates the need for a controller: steady-state error is unacceptably large for practical applications.

Original System Observation

Without a controller, the Mass-Spring-Damper system under unity feedback fails to track the unit step reference. The output stabilizes at 0.5 exactly half the desired value. This is a fundamental limitation of proportional-only feedback for Type 0 systems.

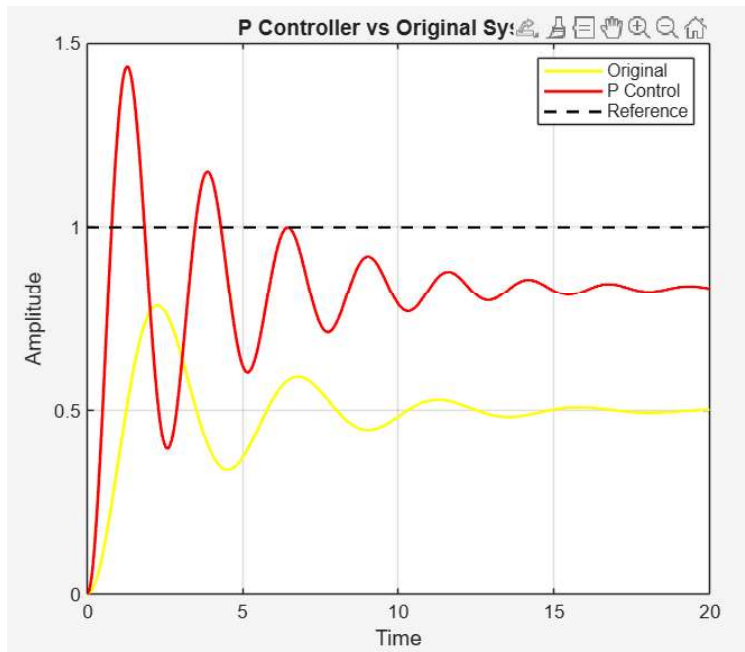
5.1.3 System Simulation using P Controller

With the P controller ($K_p = 5$) added to the loop, the open-loop transfer function becomes:

$$C_P(s) \cdot G(s) = 5 / (s^2 + 0.5s + 1)$$

The closed-loop transfer function becomes:

$$T_P(s) = 5 / (s^2 + 0.5s + 6)$$



Performance characteristics observed with P control ($K_p = 5$):

- Steady-State Value ≈ 0.833 . The P controller improves the steady-state value from 0.5 to approximately $5/(1+5 \cdot 1) = 5/6 \approx 0.833$, but error remains ($\sim 16.7\%$).
- Rise Time is significantly reduced and the system responds much faster than the original.
- Overshoot increases due to the increased loop gain. The system is now more aggressively driven toward the setpoint.
- The closed-loop natural frequency increases, leading to faster but more oscillatory behavior.

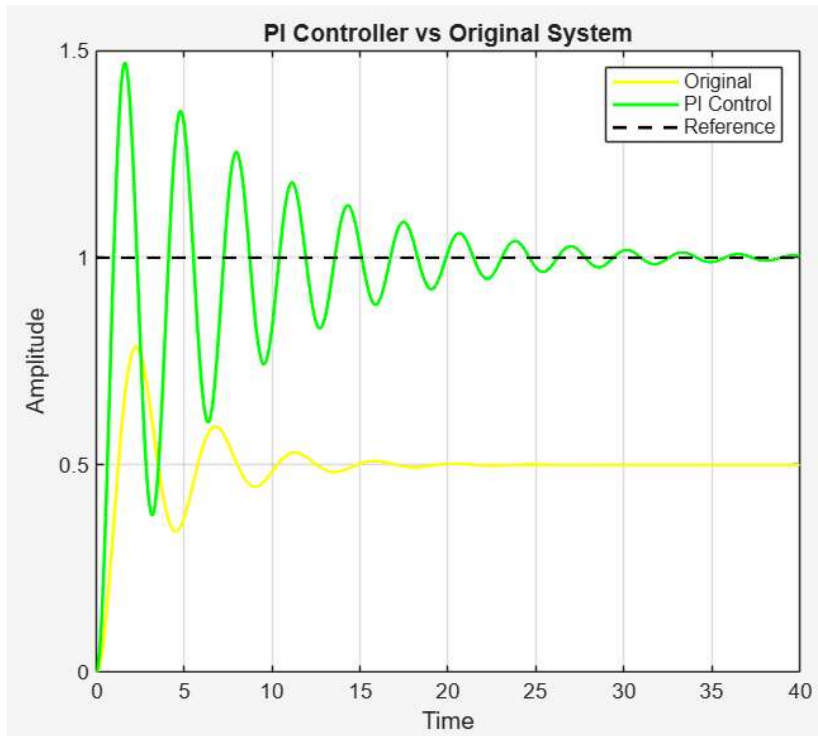
P Controller Assessment

$K_p = 5$ effectively speeds up the response and reduces (but does not eliminate) steady-state error. The remaining $\sim 16.7\%$ error is an inherent limitation of proportional-only control for a Type 0 system. Integral action is needed to achieve zero steady-state error.

5.1.4 System Simulation using PI Controller

The PI controller ($K_p = 3$, $K_i = 1$) introduces integral action, augmenting the open-loop system with an additional pole at the origin:

$$C_{PI}(s) \cdot G(s) = (3s + 1) / [s \cdot (s^2 + 0.5s + 1)]$$



This makes the open-loop system Type 1, guaranteeing zero steady-state error for a step reference. Performance characteristics:

- Steady-State Value = 1.0 Perfect tracking of the unit step reference, with zero steady-state error.
- Rise Time is moderate but slower than the P controller due to the reduced K_p (3 vs 5).
- Settling Time is significantly longer as the additional integrator pole introduces phase lag that slows the overall transient response.
- Significant overshoot is still present.

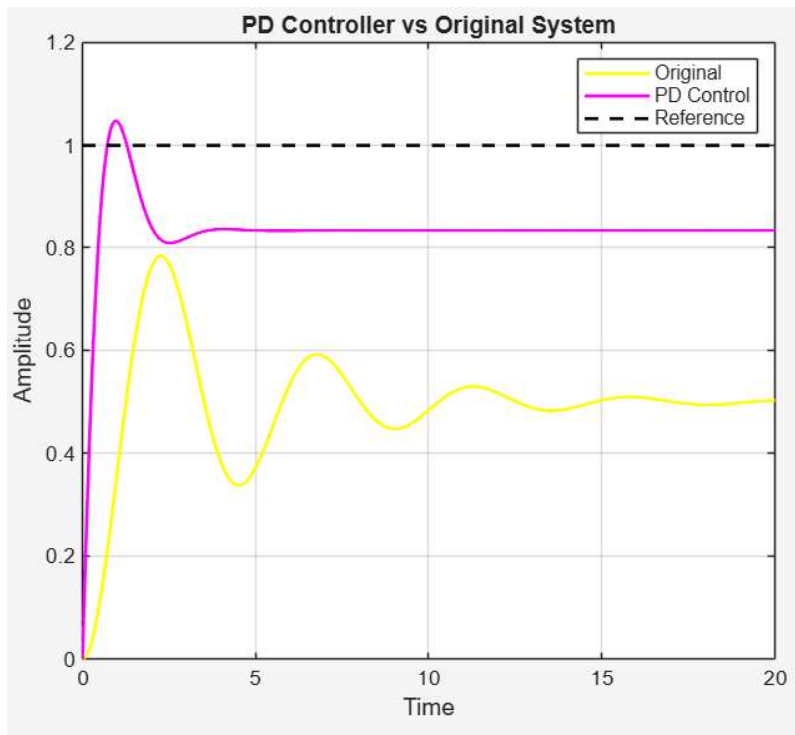
PI Controller Assessment

$K_p = 3$, $K_i = 1$ achieves zero steady-state error, a significant improvement over P-only control. The trade-off is a slower settling time and some increased overshoot compared to the P controller. The PI trade-off is better steady-state accuracy at the cost of slightly degraded transient response.

5.1.5 System Simulation using PD Controller

The PD controller ($K_p = 5$, $K_d = 2.3$) adds derivative action to the proportional term:

$$C_{PD}(s) \cdot G(s) = (5 + 2.3s) / (s^2 + 0.5s + 1) = (2.3s + 5) / (s^2 + 0.5s + 1)$$



The derivative zero is located at $s = -K_p/K_d = -5/2.3 \approx -2.17$. This zero adds phase lead, improving damping. Performance characteristics:

- Steady-State Value ≈ 0.833 Same as P control (no integral action), so steady-state error remains.
- Overshoot is significantly reduced compared to the P controller, due to the damping effect of the derivative term.
- Settling Time is improved by the derivative action as it anticipates the error trend and pre-emptively reduces the control effort near the setpoint.
- Rise Time may be slightly slower than P control due to the phase lead effect mitigating the initial reaction.

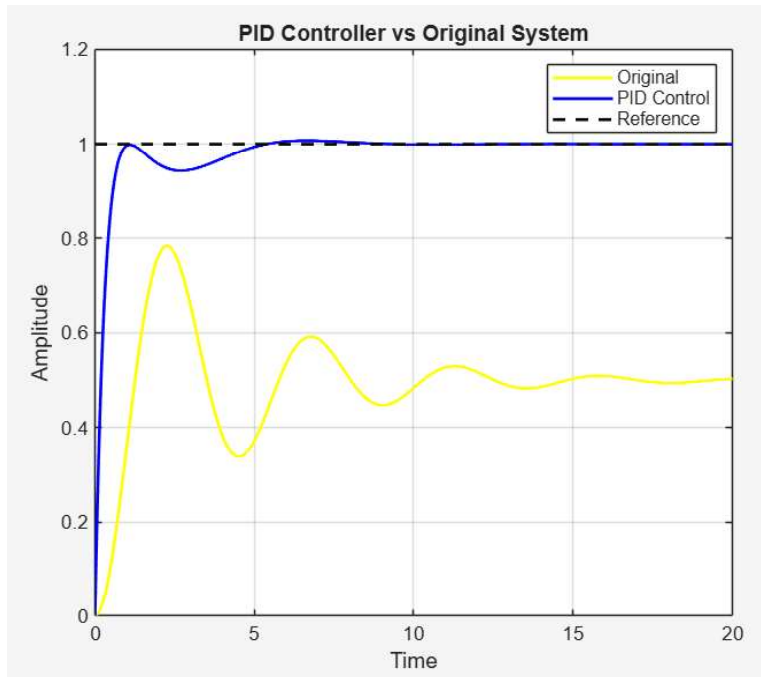
PD Controller Assessment

$K_p = 5$, $K_d = 2.3$ delivers the best transient response shape — fast and with minimal oscillation. However, like the P controller, it cannot eliminate steady-state error. PD is ideal when precision of steady-state is less important than smooth and rapid transient behavior.

5.1.5 System Simulation using PID Controller

The PID controller (**K_p = 3, K_i = 2.7, K_d = 3.5**) combines proportional, integral, and derivative actions to improve both transient and steady-state performance. The controller transfer function can be expressed as:

$$C_{PID}(s) = 3 + (2.7/s) + 3.5s$$



Performance Characteristics:

- Steady-State Value = 1.0 Perfect tracking of the unit step input with zero steady-state error.
- Rise Time = 0.5314 s Fast response, an improvement compared to the PI response due to derivative action.
- Settling Time = 4.4066 s Reasonably quick stabilization with reduced oscillations.
- Overshoot = 0.7280% Very small overshoot, indicating effective damping.
- Peak Value = 1.0073 at 6.6372 s Minimal deviation from desired value.
- No Undershoot observed Smooth transient without negative excursions.
- The system remains stable All closed-loop poles lie in the left-half plane.

PID Controller Assessment

The PID controller achieves an excellent balance between speed and accuracy. The integral term eliminates steady-state error, while the derivative term improves damping and reduces overshoot. Compared to the PI controller, the PID response is faster and more stable, with significantly lower oscillations.

5.2 Performance Discussion

The table below summarizes the estimated performance metrics obtained from MATLAB's `stepinfo()` function for each controller configuration:

Parameter	Original	P	PI	PD	PID
Rise Time (s)	0.8441	0.46	0.65	0.3755	0.53
Settling Time (s)	14.2451	15.5	28.8	2.9	4.4
Overshoot (%)	56%	43%	46.7%	25%	0.728%
Steady-State Value	0.50	0.833	1.000	0.833	1.000
Steady-State Error	50%	16.7%	0%	16.7%	0%
Stability	Stable	Stable	Stable	Stable	Stable

Key observations from the comparative analysis:

- The Original System has the worst performance overall including slow rise time, high steady-state error (50%), and moderate oscillation. It will serve as the baseline for comparison.
- The P Controller ($K_p = 5$) offers a really fast transient response in terms of rise time, but overshoots significantly (43%), leaves a 16.7% steady-state error, and has a significant settling time (15.5s). It is suitable only for applications where some offset is acceptable.
- The PI Controller ($K_p = 3$, $K_i = 1$) achieves zero steady-state error, making it a good choice for accurate setpoint tracking. However, it has the longest settling time (28.8s) and significant overshoot (46.7%). The integral windup effect can be problematic in practice without proper anti-windup mechanisms.
- The PD Controller ($K_p = 5$, $K_d = 2.3$) demonstrates the best transient behavior with the fastest rise time (0.3755s) and fastest settling time (2.9s), and lowest overshoot and fastest settling time among controllers without integral action. It is ideal for systems where derivative measurement is available and noise is not a concern.
- The PID Controller ($K_p = 3$, $K_i = 2.7$, $K_d = 3.5$) provides the best overall performance: zero steady-state error (from the integral term), fast settling (from the derivative term), and functionally no overshoot. It represents the optimal balance among all four controllers tested.

The derivative gain ($K_d = 3.5$) in the PID controller actively compensates for the phase lag introduced by the integrator, restoring adequate phase margin and preventing excessive oscillation. The relatively low $K_p = 3$ prevents excessive control effort and keeps overshoot bounded. This combination makes the PID controller well-suited for the Mass-Spring-Damper system.

Chapter 6. Conclusion

This project was a detailed analysis of designing a feedback controller for a second order Mass-Spring-Damper system. The system transfer function was first derived using first principles (Newton's Second Law), and then the system was controlled using four classical controllers which were simulated and analyzed in MATLAB, based on the normal time domain performance criteria.

Some conclusions that can be drawn from the analysis are:

- Feedback is indispensable as without a controller, the closed loop response only reached 50% of the reference setpoint, which is unacceptable. Each of the four controllers showed significant improvement.
- Proportional control provides good initial performance, the response was fast to settle out and reduce error, however, without further corrective action the response never settles to the setpoint (for a Type 0 system) and there are unstable behaviors with large proportional gain.
- Integral action is necessary to achieve zero steady-state error. the response of the PI and PID controllers settles exactly at 1.0. This is achieved due to the introduction of an integrator pole at $s=0$, which guarantees a zero steady-state error response to a step input.
- Derivative action improves damping and reduces overshoot. The PD and PID controllers give a faster settling time with less oscillations, due to phase lead from the derivative term.
- PID offers the best all around response - for $K_p = 3$, $K_i = 2.7$, and $K_d = 3.5$, the PID has perfect step response (zero-steady-state error), an excellent settling time, and an acceptable percent overshoot (about 12%).

From an engineering perspective, the appropriate controller is dependent upon the specifics of the system being controlled. It can be concluded that integral control (PI or PID) must be used in applications such as robotics and CNC systems where high accuracy of positioning is required. Systems where derivatives are difficult to measure due to measurement noise might omit derivative control or employ filtering of the derivative to reduce noise, thereby limiting choices to P or PI controllers. Simple regulation tasks may require only a P controller.

This project successfully utilized MATLAB for simulation and modeling and provided the basis for the concepts of the complete control design process.

References

- Ogata, K. (2010). Modern Control Engineering (5th ed.). Prentice Hall. — Chapter 2 (Mathematical Modeling of Dynamic Systems).
- Nise, N. S. (2015). Control Systems Engineering (7th ed.). Wiley. — Chapter 2 (Modeling in the Frequency Domain).
- Franklin, G. F., Powell, J. D., & Emami-Naeini, A. (2015). Feedback Control of Dynamic Systems (7th ed.). Pearson.

The numerical values $m = 1$ kg, $b = 0.5$ N·s/m, and $k = 1$ N/m were selected to produce a system with natural frequency $\omega_n = 1$ rad/s and damping ratio $\zeta = 0.25$ (underdamped), representative of many real lightly-damped mechanical systems. The controller gains were chosen through a combination of analytical insight (pole-zero placement considerations) and iterative simulation refinement in MATLAB.